Advanced DBMS

CS-6418 — Complete Study Notes

Units I – V · M.Tech (IA&SE) · SCSIT, DAVV

With ER Diagrams · Workflows · Use Cases · Schema Tables

Crow's Foot Notation · Textbook-Style Visual Reference

Unit	Topic	Key Concepts
I	**DBMS Fundamentals & ER Model**	Keys, Entities, Relationships, Schema Reduction
II	**Relational Algebra, Calculus & SQL**	σ π ■, TRC/DRC, SQL Deep Dive
III	**Functional Dependencies & Normalization**	1NF→5NF, BCNF, Triggers, Integrity
IV	**Transactions, Concurrency & Recovery**	ACID, Serializability, WAL, Query Optimization
V	**Graph, Spatial, Temporal & NoSQL**	CAP Theorem, 4 NoSQL Types, Valid/Tx Time

Notation used in diagrams

Symbol	Meaning	Colour
`PK`	Primary Key — uniquely identifies each row	**Blue**
`FK`	Foreign Key — references primary key of another table	**Red**
`PK, FK`	Composite key and foreign key (junction tables / weak entities)	**Purple**
`Double border`	Weak entity — depends on owner for identification	**Purple border**
`Green header`	Junction/associative table (resolves M:N relationship)	**Teal**

UNIT I — DBMS Fundamentals & ER Model

8 Hours · ER Diagrams, Keys, Schema Reduction, Specialization/Generalization

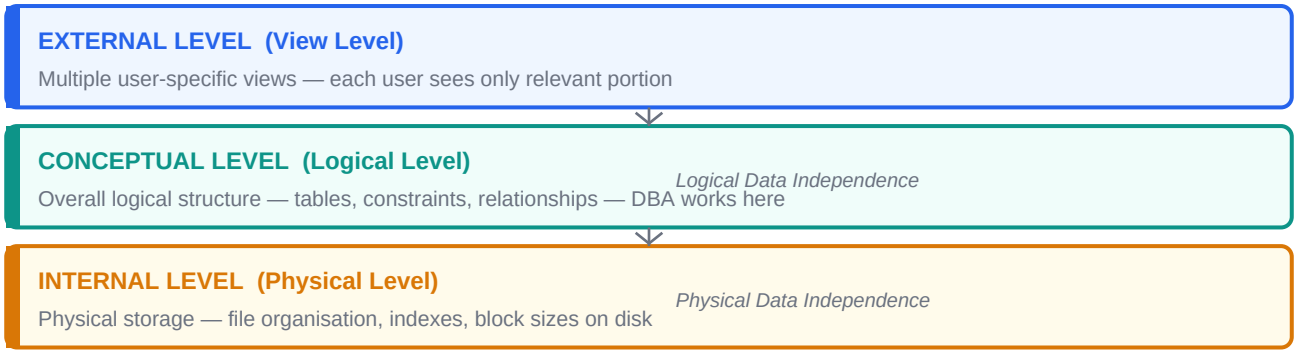
1.1 Advantages of DBMS over File Processing

A DBMS is software that provides a **controlled interface** between users/applications and the underlying data, eliminating all problems of the older file-processing approach.

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '**File-System Problem**' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='File-System Problem', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '**DBMS Solution**' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='DBMS Solution', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Data redundancy & inconsistency' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=8.5, greek=0, italic=0, link=[], rise=0, text='Data redundancy ', textColor=Color(.066667,.094118,.152941,1), us_lines=[]), ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=8.5, greek=0, italic=0, link=[], rise=0, text='&', textColor=Color(.066667,.094118,.152941,1), us_lines=[]), ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=8.5, greek=0, italic=0, link=[], rise=0, text=' inconsistency', textColor=Color(.066667,.094118,.152941,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Centralised storage — one copy, consistent everywhere' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=8.5, greek=0, italic=0, link=[], rise=0, text='Centralised storage — one copy, consistent everywhere', textColor=Color(.066667,.094118,.152941,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'No ad-hoc querying' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=8.5, greek=0, italic=0, link=[], rise=0, text='No ad-hoc querying', textColor=Color(.066667,.094118,.152941,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'SQL query language — no new programs needed per query' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=8.5, greek=0, italic=0, link=[], rise=0, text='SQL query language — no new programs needed per query', textColor=Color(.066667,.094118,.152941,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Data isolation across files' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=8.5, greek=0, italic=0, link=[], rise=0, text='Data isolation across files', textColor=Color(.066667,.094118,.152941,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Unified relational/logical data model' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=8.5, greek=0, italic=0, link=[], rise=0, text='Unified relational/logical data model', textColor=Color(.066667,.094118,.152941,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Scattered integrity checks' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=8.5, greek=0, italic=0, link=[], rise=0, text='Scattered integrity checks', textColor=Color(.066667,.094118,.152941,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Constraints declared once, enforced by DBMS' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=8.5, greek=0, italic=0, link=[], rise=0, text='Constraints declared once, enforced by DBMS', textColor=Color(.066667,.094118,.152941,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'No atomicity for partial failures' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=8.5, greek=0, italic=0, link=[], rise=0, text='No atomicity for partial failures', textColor=Color(.066667,.094118,.152941,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Transaction management (ACID — Unit IV)' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=8.5, greek=0, italic=0, link=[], rise=0, text='Transaction management (ACID — Unit IV)', textColor=Color(.066667,.094118,.152941,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Concurrent-access anomalies' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=8.5, greek=0, italic=0, link=[], rise=0, text='Concurrent-access anomalies', textColor=Color(.066667,.094118,.152941,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Concurrency control (locks, timestamps — Unit IV)' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=8.5, greek=0, italic=0, link=[], rise=0, text='Concurrency control (locks, timestamps — Unit IV)', textColor=Color(.066667,.094118,.152941,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Coarse-grained security' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=8.5, greek=0, italic=0, link=[], rise=0, text='Coarse-grained security', textColor=Color(.066667,.094118,.152941,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Authorization at relation / view / attribute level' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=8.5, greek=0, italic=0, link=[], rise=0, text='Authorization at relation / view / attribute level', textColor=Color(.066667,.094118,.152941,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph

1.2 Three-Schema Architecture & Data Independence



Physical DI — change internal schema without touching conceptual/external. *Logical DI* — change conceptual schema without touching external views/apps. **Logical DI is harder to achieve** because apps are tightly coupled to logical structure.

1.3 Keys in the Relational / ER Model

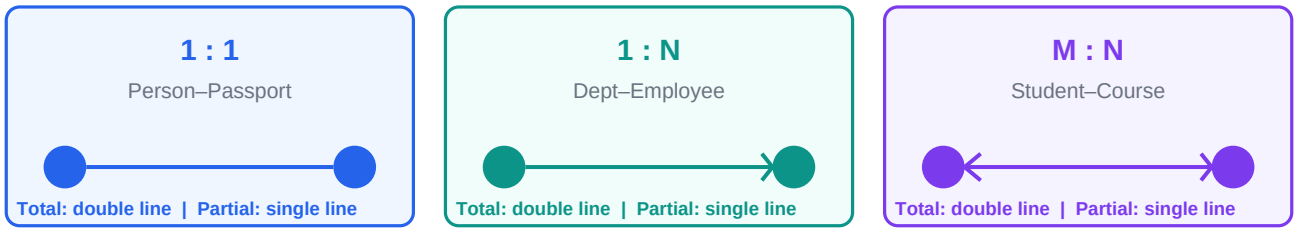
Key Type	Definition
Super Key	Any attribute set that uniquely identifies a tuple. May contain redundant attributes.
Candidate Key	Minimal super key — removing any attribute breaks uniqueness. Every CK is a SK, not vice versa.
Primary Key	The candidate key chosen as the principal identifier. Cannot be NULL.
Alternate Key	Candidate keys not chosen as PK — still unique, still minimal.
Foreign Key	Attribute(s) in one table referencing the PK of another (or same) table.

Exam Tip
Super Key ⊇ Candidate Key ⊇ {Primary Key}. "Every CK is a SK but every SK need not be a CK" is a classic 2-mark question.

1.4 ER Model — Concepts & Types of Attributes

Attribute Type	Description / Example	ER Symbol
Simple / Atomic	Cannot be divided — e.g. roll_no	Solid oval
Composite	Can be split — e.g. Name → (First, Last)	Oval with sub-ovals
Single-valued	One value per entity — e.g. date_of_birth	Plain oval
Multi-valued	Multiple values — e.g. phone_numbers	Double oval
Derived	Computed from another — e.g. age from dob	Dashed oval
Key	Uniquely identifies entity — e.g. roll_no	Underlined name

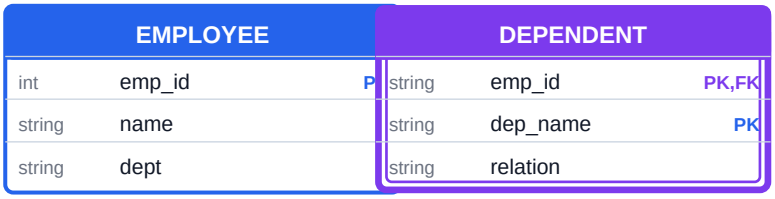
1.5 Mapping Cardinalities & Participation Constraints



Total participation — every entity MUST participate (double line). **Partial participation** — some entities may not participate (single line).

1.6 Strong vs Weak Entity Sets

A **weak entity** has no primary key of its own — it is identified by combining its **partial key (discriminator)** with the PK of its **owner (identifying) entity**. Shown with a **double rectangle** and a double diamond for the identifying relationship.



DEPENDENT is uniquely identified by (emp_id, dep_name) — two different employees could each have a dependent named "Raju", so dep_name alone is NOT a key.

1.7 Specialization, Generalization & Aggregation

Concept	Definition	Example
Specialization	Top-down: start from a general entity, divide into specialised sub-sets based on distinguishing characteristics.	Employee → Manager, Engineer
Generalization	Bottom-up: start from multiple specific entities, combine common features into one higher-level entity.	Car + Truck → Vehicle
Aggregation	Treat a relationship as a higher-level entity so it can itself participate in another relationship.	(Employee–Project)Works_On → further related to Manager
Inheritance	Sub-entities inherit all attributes and relationships of their super-entity (like OOP inheritance).	Engineer inherits emp_id, name from Employee

1.8 ER Schema Reduction to Tables (All 6 Rules)

Rule	Mapping Strategy
1. Strong Entity	Becomes a table; composite attributes → individual columns; multi-valued attribute → separate table with entity PK + the attribute.
2. Weak Entity	Table includes all its own attributes + PK of owner as FK; PK = owner PK + partial key.
3. 1:1 Relationship	Include PK of one side as FK in the other's table — prefer the side with total participation.
4. 1:N Relationship	Include PK of the "1" side as FK in the table of the "N" side.
5. M:N Relationship	Create a separate junction table with PKs of both sides as composite PK + any relationship attributes.
6. Specialization / Generalization	(a) Single table with discriminator column, or (b) Sub-class tables with inherited PK only, or (c) Sub-class tables duplicating all inherited attributes.

UNIT II — Relational Algebra, Calculus & SQL

8 Hours · σ π ■ operators, TRC vs DRC, SQL deep-dive

2.1 Relational Algebra — Fundamental Operations

Relational algebra is a **procedural** query language — it describes step-by-step *how* to compute the result using operators that take relations as input and produce a relation as output.

Operation	Notation Example	Meaning
Select σ	σ(cgpa>8)(Student)	Filters rows satisfying a predicate
Project π	π(name,dept)(Student)	Selects specific columns; removes duplicates
Union ∪	R ∪ S	All tuples from R and S; removes duplicates; union-compatible required
Set Difference –	R – S	Tuples in R but not in S; union-compatible required
Cartesian Product ×	R × S	Every tuple of R paired with every tuple of S
Rename ρ	ρ(S, Student)	Renames relation and/or attributes
Natural Join ■	R ■ S	Equijoin on all same-named attributes; duplicate column removed
Division ÷	R ÷ S	"Find X related to ALL Y" — e.g. students enrolled in all courses

Join Types — Comparison

Join Type	Condition	Behaviour	Note
Theta Join ■θ	Any condition (=, <, >, ≠)	σθ(R × S)	Most general
Equijoin	Only equality (=)	Like theta but θ uses = only	Keeps duplicate join column
Natural Join ■	Auto-equijoin on same-named attrs	Removes duplicate column	Cleaner output
Left Outer ■	All rows from LEFT + NULLs	Retains unmatched left tuples	Common in SQL
Right Outer ■	All rows from RIGHT + NULLs	Retains unmatched right tuples	
Full Outer ■	All rows from BOTH + NULLs	Union of both outer joins	FULL OUTER JOIN in SQL

2.2 Relational Calculus

Type	Syntax	Key Idea	Example
Tuple RC (TRC)	{t | P(t)}	Variables range over whole tuples	{t | t∈ Student ∧ t.cgpa>8}

Type	Syntax	Key Idea	Example
Domain RC (DRC)	{ | P(…)}	Variables range over attribute values (domains)	{ | ∃r(r∈ Student ∧ r.name=n ∧ r.cgpa=c ∧ c>8)}

RA vs RC
Relational Algebra = PROCEDURAL (how). Relational Calculus = DECLARATIVE (what). Both are relationally complete — equivalent in expressive power (every RA expression has a RC equivalent and vice versa).

2.3 SQL Deep-Dive

Logical Order of Execution

Although SELECT is written first syntactically, it is evaluated 5th — this is why a SELECT alias cannot be used in WHERE or HAVING of the same query.

WHERE vs HAVING

WHERE filters individual rows BEFORE grouping — cannot use aggregate functions. HAVING filters groups AFTER GROUP BY — specifically for aggregate conditions like HAVING AVG(cgpa)>8.

Correlated Subquery

Inner query references a column from the outer query and is re-evaluated for EACH outer row. Non-correlated subquery is independent and evaluated only once.

NULL Handling

Any arithmetic on NULL → NULL. Comparisons with NULL → UNKNOWN (not TRUE/FALSE). Always use IS NULL / IS NOT NULL, never = NULL.

UNION vs UNION ALL

UNION removes duplicates (slower). UNION ALL retains all rows (faster — skip deduplication).

DELETE vs TRUNCATE vs DROP

Command	Category	Behaviour	Structure	Rollback
DELETE	DML	Row by row; can use WHERE; slower	Table structure kept	Can be rolled back
TRUNCATE	DDL	All rows at once; no WHERE; faster	Table structure kept	Often NOT rollback-able
DROP	DDL	Entire table removed	Table structure GONE	Cannot be rolled back

UNIT III — Functional Dependencies, Normalization & Integrity

8 Hours · Armstrong's Axioms · 1NF→5NF · BCNF · Triggers

3.1 Functional Dependencies

X → Y means "knowing X uniquely determines Y". For any two tuples t1, t2 in R: if t1.X = t2.X then t1.Y = t2.Y must hold.

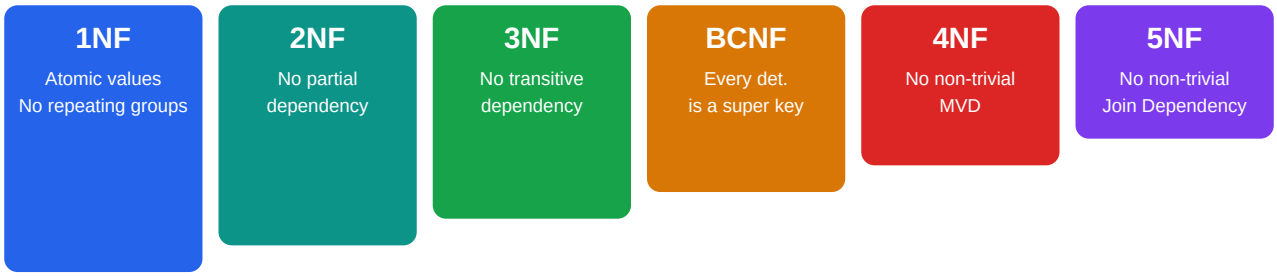
Armstrong's Axioms (must memorise)

Axiom	Rule	Explanation
Reflexivity	If Y ⊆ X, then X → Y	Trivial dependency — always true
Augmentation	If X → Y, then XZ → YZ	Adding same attributes to both sides preserves FD
Transitivity	If X→Y and Y→Z, then X→Z	The most useful — basis for 3NF violation detection
Union (derived)	If X→Y and X→Z, then X→YZ	Combine RHS
Decomposition (derived)	If X→YZ, then X→Y and X→Z	Split RHS

Attribute Closure X■ — Algorithm

1. Start: result = X. 2. For each FD A→B in F: if A ⊆ result, add B to result. 3. Repeat until no change. 4. result = X■. **Use:** X is a candidate key if X■ = all attributes and no proper subset of X has this property.

3.2 Normal Forms — Visual Summary



Each normal form is a stricter superset of the previous — every BCNF relation is also in 3NF, etc.

NF	Condition Removed	Key Note
1NF	All values atomic; no multi-valued/composite/repeating groups	—
2NF	1NF + no partial dependency (non-prime attr depends on ENTIRE composite PK)	Only relevant when PK is composite
3NF	2NF + no transitive dependency (non-prime attr depends via another non-prime attr)	Formal: every non-trivial FD X→A has X=superkey OR A=prime attr
BCNF	Every non-trivial FD X→A must have X as a super key (no exception for prime attrs)	Stricter than 3NF; may sacrifice dependency preservation

NF	Condition Removed	Key Note
4NF	BCNF + no non-trivial MVD (X →→ Y) unless X is a super key	Removes multi-valued dependency redundancy
5NF	4NF + every join dependency implied by candidate keys	Theoretical; rarely applied in practice

3NF vs BCNF — Key Exam Comparison

3NF	BCNF
Allows X→A when A is prime attr (part of any CK)	Never — X must ALWAYS be a super key
Always achievable with dependency preservation	May require sacrificing dependency preservation
Weaker — more permissive	Stricter — every BCNF is also in 3NF, not vice versa

3.3 The Three Anomalies (with Example)

Consider unnormalised **StudentCourse(roll_no, name, course_id, course_name, instructor)**:

Anomaly	Description
Insertion Anomaly	Cannot insert a new course unless at least one student is already enrolled (course info lives in student rows).
Deletion Anomaly	Deleting the only student enrolled in a course loses all info about that course as a side-effect.
Update Anomaly	Changing an instructor's name must be done in EVERY row for that course — missing one row leaves inconsistent data.

Fix
Decompose into Student(roll_no, name), Course(course_id, course_name, instructor), Enrollment(roll_no, course_id) — each fact stored in exactly one place.

3.4 Database Integrity

Integrity Mechanism	Description
Entity Integrity	No part of a primary key can be NULL — every tuple must be uniquely identifiable.
Referential Integrity	FK value must match an existing PK in the referenced table, or be entirely NULL.
Domain Constraint	Attribute values must fall within the defined domain — e.g. cgpa BETWEEN 0 AND 10.
CHECK Constraint	Custom boolean condition on a column — e.g. age >= 18.
Assertion	General constraint spanning multiple tables — checked on every modification.
Trigger	Stored procedure auto-executed on INSERT/UPDATE/DELETE (Event–Condition–Action model).

Referential Integrity Actions (ON DELETE / ON UPDATE)

Action	Behaviour
CASCADE	Auto delete/update matching child rows when parent is deleted/updated.
SET NULL	Set FK column to NULL in child table instead of deleting/blocking.
RESTRICT / NO ACTION	Reject the parent delete/update if matching child rows still exist.
SET DEFAULT	Set FK to a predefined default value.

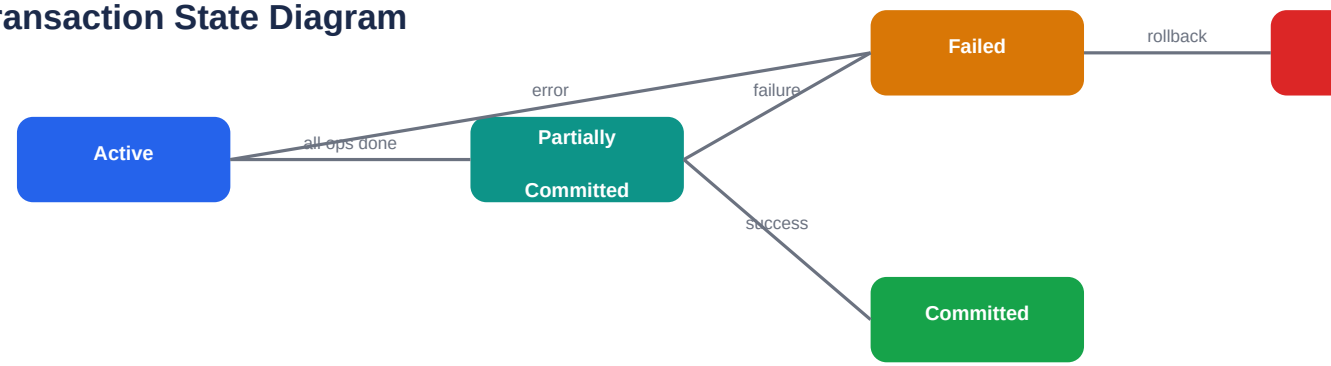
UNIT IV — Transactions, Concurrency & Recovery

8 Hours · ACID · Serializability · WAL · Deadlock · Query Optimization

4.1 ACID Properties

Property	Definition	Classic Example
Atomicity	All-or-nothing — if a transaction fails mid-way, ALL its effects are rolled back (undone) completely.	Fund transfer: either BOTH debit and credit happen, or NEITHER does.
Consistency	Transaction takes DB from one consistent state to another — all integrity constraints remain satisfied.	Total money in bank unchanged after a transfer between accounts.
Isolation	Intermediate state of one transaction is NOT visible to other concurrent transactions.	Two users booking last seat — only one succeeds; neither sees partial state of the other.
Durability	Once a transaction commits, its changes persist permanently — even after a crash.	After a successful payment, the record stays even if the server crashes seconds later.

4.2 Transaction State Diagram



4.3 Concurrency Problems

Problem	Description
Lost Update	T1 and T2 both read X; T1 writes back; T2 overwrites T1's update silently.
Dirty Read	T1 reads a value written by T2 which has not yet committed; T2 aborts → T1 read a value that never existed.
Unrepeatable Read	T1 reads X twice; T2 updates and commits X between the two reads → T1 gets different values.
Incorrect Summary	T1 computes SUM while T2 is mid-transfer; T1 may count some accounts before and some after the transfer.

4.4 Serializability & Precedence Graph

A schedule is **conflict-serialisable** if it can be transformed into a serial schedule by swapping non-conflicting operations. Test: draw a **precedence graph** — add edge Ti→Tj if Ti has a conflicting operation before Tj on the

same data item. **No cycle** = conflict-serialisable.

4.5 Recovery — Write-Ahead Logging (WAL)

WAL Rule (Critical)
A log record MUST be written to stable storage BEFORE the corresponding database modification is written to disk. This ensures crash recovery is always possible.

Scheme	Mechanism	Recovery Ops	Trade-off
Deferred Modification	DB updated only AFTER commit; log records applied at commit.	Only REDO needed	Needs more memory
Immediate Modification	DB updated during execution; log record written first (WAL).	Both UNDO and REDO needed	Widely used in practice

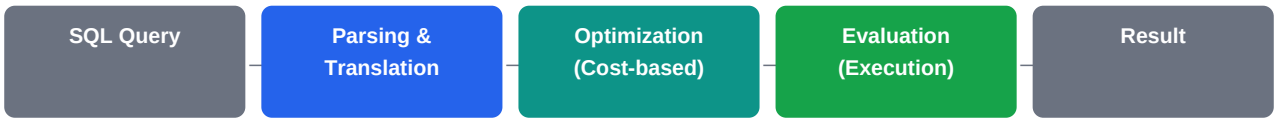
UNDO vs REDO

Operation	Purpose
UNDO(Ti)	Restores data items to their OLD values (before Ti ran) — used when Ti did NOT commit at crash time.
REDO(Ti)	Sets data items to their NEW values — used when Ti DID commit but changes may not be on disk yet.

4.6 Deadlock

Approach	Method
Prevention	Design system so deadlock can never happen — e.g. acquire all locks at once, or use Wait-Die / Wound-Wait timestamp schemes.
Detection & Recovery	Periodically construct a Wait-For Graph; cycle = deadlock; choose a victim transaction to abort and roll back.

4.7 Query Processing & Optimization



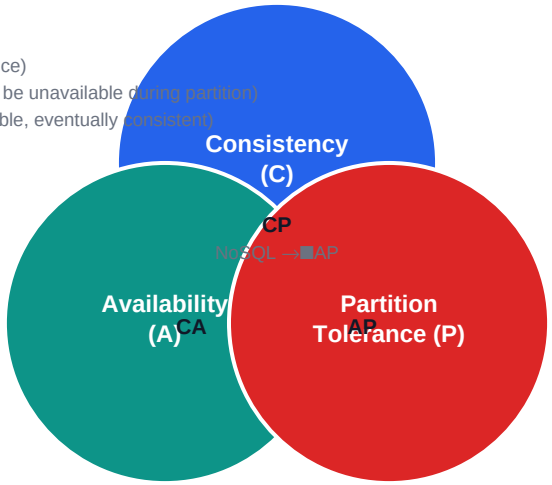
Optimization heuristics: (1) Push σ (selections) as early as possible to reduce tuple count. (2) Combine Cartesian products + subsequent σ into a join. (3) Use most selective conditions first, use indexes wherever available.

UNIT V — Graph, Spatial, Temporal Databases & NoSQL

CAP Theorem · 4 NoSQL Types · Valid vs Transaction Time · Graph/Spatial Indexing

5.1 CAP Theorem

CA: traditional RDBMS (no partition tolerance)
CP: MongoDB, HBase (consistent but may be unavailable during partition)
AP: Cassandra, DynamoDB (always available, eventually consistent)



Key Point
In real distributed systems, network partitions are unavoidable → P is always required. So the real choice is between C (CP system) and A (AP system).

5.2 NoSQL Database Types

Type	Data Model	Examples	Best For
Key-Value Store	Simple key→value pairs; value is opaque to DB	Redis, DynamoDB, Riak	Caching, sessions, fast lookups
Document Store	JSON/BSON documents; flexible/nested schema per document	MongoDB, CouchDB	CMS, catalogs, evolving data
Column-Family	Rows with varying columns organised into column families; sparse	Cassandra, HBase	Analytics, time-series, write-heavy
Graph Database	Nodes + edges + properties; index-free adjacency for traversal	Neo4j, ArangoDB	Social networks, fraud, recommendations

5.3 SQL vs NoSQL Comparison

Aspect	SQL (Relational)	NoSQL
Schema	Fixed, predefined	Dynamic/flexible — different records can differ
Data Model	Tables with rows & columns	Key-value, Document, Column-family, Graph
Scaling	Vertical (bigger machine)	Horizontal (more machines)
Consistency	Strong ACID	Often Eventual (BASE model)

Aspect	SQL (Relational)	NoSQL
Query Language	SQL (standardised)	Varies — MQL, CQL, Gremlin, Cypher…
Best For	Structured data, complex joins, transactions	Large volume, unstructured/semi-structured, high write throughput

ACID vs BASE

ACID	BASE
Atomicity — all or nothing	Basically Available — always responds (may be stale)
Consistency — valid state always	Soft State — state may change without input (propagation)
Isolation — transactions independent	Eventually Consistent — all nodes converge given enough time
Durability — committed data persists	(Priority: availability + partition tolerance over strict consistency)

5.4 Temporal Databases

Type	Definition	Example
Valid Time	When a fact is TRUE in the REAL WORLD — independent of when it was recorded.	Salary raise effective 1-Jan entered on 15-Jan → valid time starts 1-Jan
Transaction Time	When a fact was RECORDED in the DATABASE as the current known value.	Same example → transaction time starts 15-Jan (when DB was updated)
Bitemporal	Tracks BOTH valid time and transaction time simultaneously — most complete form.	Needed for auditing: "what did we believe on date X about event at date Y?"

5.5 Graph & Spatial Databases

Concept	Description	Example / Use
Graph DB Nodes	Entities with labels and key-value properties	Person {name:"Rohit", age:25}
Graph DB Edges	Directed, named relationships with properties	(Rohit)-[:FOLLOWS]->(Priya)
Index-free Adjacency	Each node stores direct pointers to its neighbours — traversal cost O(1) per hop	No joins needed for graph walks
Spatial: R-tree	Hierarchical grouping of nearby objects using bounding rectangles	Range/nearest-neighbour queries
Spatial: Quad-tree	Recursive division of 2D space into 4 quadrants	Point indexing, GIS
Spatial Query Types	Range, nearest-neighbour, topological (contains, intersects, overlaps), distance	Google Maps, ride-hailing

Quick Exam-Day Revision Checklist

Tick off each item before entering the exam hall

Unit I

- ■ Advantages of DBMS over file system — all 7 points
- ■ Three-schema architecture diagram + physical vs logical data independence
- ■ Super Key → Candidate Key → Primary Key hierarchy (minimality condition)
- ■ Weak entity: double rectangle, double diamond, partial key (dashed underline)
- ■ Specialization (top-down) vs Generalization (bottom-up) — inverse processes
- ■ All 6 rules for ER schema reduction to tables

Unit II

- ■ All 6 fundamental RA operators with symbols and examples
- ■ Union-compatibility rule for ∪, −, ∩
- ■ Theta Join vs Equijoin vs Natural Join (differences)
- ■ RA (procedural) vs RC (declarative) — relational completeness equivalence
- ■ SQL logical execution order: FROM→WHERE→GROUP BY→HAVING→SELECT→ORDER BY
- ■ WHERE vs HAVING; correlated vs non-correlated subquery
- ■ DELETE vs TRUNCATE vs DROP comparison table

Unit III

- ■ Armstrong's 3 axioms: Reflexivity, Augmentation, Transitivity
- ■ Attribute closure X■ algorithm — 4 steps
- ■ Three anomalies with example (Insertion, Deletion, Update)
- ■ 1NF → 2NF → 3NF → BCNF → 4NF → 5NF — condition for each
- ■ 3NF vs BCNF — when each can be achieved with dependency preservation
- ■ Entity Integrity Rule + Referential Integrity Rule
- ■ Trigger: Event–Condition–Action; BEFORE/AFTER; ROW/STATEMENT
- ■ ON DELETE: CASCADE, SET NULL, RESTRICT

Unit IV

- ACID properties — all four with one example each
- Transaction state diagram: Active→Partially Committed→Committed/Failed→Aborted
- Four concurrency problems: Lost Update, Dirty Read, Unrepeatable Read, Incorrect Summary
- Precedence graph for conflict serializability — no cycle = serializable
- WAL rule: log record before DB modification — exact phrasing
- UNDO (not committed) vs REDO (committed, may not be on disk)
- Deferred vs Immediate DB modification — which requires UNDO+REDO
- Deadlock: Prevention vs Detection-and-Recovery (Wait-For Graph)
- Query processing steps: Parsing → Optimization → Evaluation

Unit V

- CAP theorem — C, A, P definitions; why P is non-negotiable in distributed systems
- CP vs AP trade-off — examples of each
- 4 NoSQL types with one example DB each (Key-Value, Document, Column-Family, Graph)
- SQL vs NoSQL comparison table (schema, scaling, consistency, query lang)
- ACID vs BASE — all 3 BASE properties
- Valid time vs Transaction time vs Bitemporal — with example
- Graph DB: nodes, edges, index-free adjacency
- Spatial DB: R-tree vs Quad-tree; range, NN, topological queries